An Optimal Algorithm for the Maximum Two-Chain Problem+

R. D. Lou*
M. Sarrafzadeh*
D. T. Lee*

Abstract

Given a point set ρ, a *chain* is a subset $C \subseteq \rho$ of points in which for any two points one is dominated by the other. A *two-chain* is a subset of ρ that can be partitioned into two chains. A two-chain with maximum cardinality among all possible two-chains is called a *maximum two-chain*. In this paper we present a $\Theta(n \log n)$ time and $\Theta(n)$ space algorithm for finding a maximum two-chain in a point set ρ, where $n = |\rho|$. Maximum two-chain finds applications in, for example, graph-theoretic problems, VLSI layout, and sequence manipulation.

1 Introduction

Consider a point set $\rho = \{P_1, P_2, ..., P_n\}$ on the x-y plane, where $P_i = (x_i, y_i)$ with $x_i > 0$ and $y_i > 0$; by convention, $x_i > x_j$ for $i > j$ and all the y's are distinct. Following [PS] we say P_i *dominates* P_j if $x_i > x_j$ and $y_i > y_j$. Points P_i and P_j are *crossing* if $x_i > x_j$ and $y_j > y_i$ or vice versa; otherwise, they are *noncrossing*. A subset of ρ is called a *chain* if its points are pairwise noncrossing. A subset is called a *two-chain* if it contains no three points, which are pairwise crossing. A *maximum chain* is a chain with the maximum number of points among all chains. A *maximum two-chain* is a two-chain with the maximum number of points among all two-chains. The algorithm for finding a maximum two-chain can be used to solve the following problems:

Graph-theoretic Problem : Given a permutation graph model, find a maximum bipartite subgraph. The problem of finding a maximum bipartite subgraph in an overlap graph (i.e., circle graph) is generally NP-hard [SL1]. However, for problems with a fixed "partition number," a polynomial-time algorithm is proposed in [SL2].

VLSI Layout : The two-layer topological via minimization problem in a restricted two sided routing region (called the *two-sided-channel TVM problem*) [SL1,SL2].

Sequence Manipulation : Given a sequence of numbers, find a maximum two increasing subsequence.

The maximum one chain problem can be solved by an algorithm proposed in [AK] in $\Theta(n \log n)$ time. For the maximum two-chain problem it is easy to find an example for which the "greedy method" (e.g., applying the algorithm for the maximum one chain problem twice, where the chain obtained in the first application is removed from the original set before the second application of the algorithm) will not produce a maximum two-chain [SL1]. Previously, an $O(n^2 \log n)$ time algorithm

+This work was supported in part by the National Science Foundation under Grants DCR 8420814 and MIP 8709074.
*Center for Integrated Microelectronic Systems (CIMS), The Technological Institute, Northwestern University, Evanston, IL 60208, also with Department of Electrical Engineering and Computer Science.

to solve the maximum two-chain problem was proposed [SL1] . In this paper we present a $\Theta(n \log n)$ time algorithm for finding a maximum two-chain of a point set ρ, where $n = |\rho|$ (an $\Omega(n \log n)$ lower-bound is also established).

This paper is organized as follows. In Section 2, the basic idea of the algorithm is outlined. In Section 3, details of the algorithm are given, and its time complexity is analyzed.

2 Basic Idea of the Algorithm

Consider a chain $C = \{P_{\sigma_1}, P_{\sigma_2}, ..., P_{\sigma_{|C|}}\}$, where, by convention, $x_{\sigma_i} < x_{\sigma_j}$ for $i < j$. Point P_{σ_i} is called the *i-th point* of the chain. The first point and the last point are also called the *bottom* and the *top* of the chain, respectively. Given a set of points ρ, the points which are not dominated by any other points are called the *maxima* of ρ. We can find the maxima of ρ : all maxima are at the same "*level*", known as the *dominance hull* [PS] of ρ. Then ignore the points at this level and recursively find other points at the same level for the remaining points. This process is repeated until all points in ρ have been assigned a level (Figure 1). We call the levels from bottom to top *levels 1, 2, ... , s*, where level s contains points on the dominance hull of ρ. We denote the set of points at levels i by ρ_i. We also assign each point $P_j \in \rho_i$ a value $L(j) = i$, called *L-value* of point P_j, to indicate that P_j is at level $L(j) = i$. The set of points in $\cup_{i=u}^{v} \rho_i$, where $u \leq v$, is denoted by $\rho_{u,v}$. By convention, $\rho_{u,v}=\phi$, for $u > v$. The levels have a property stated in Lemma 1.

Lemma 1 *Each point at level u, where* $1 \leq u < s$, *is dominated by at least one point at level* $u + 1$.

The point with the smallest (or largest) x-coordinate at a level is called the *left-* (or *right-*) *point* of the level. The point at level $u+1$ with the smallest (or largest) x-coor-dinate among all points dominating P_a at level u is called the *left-dominating* (or *right-dominating*) *point* of P_a. Let the left-point of level 1 be the first point of a chain, and let the left-dominating point of the i-th point of the chain be the $(i + 1)$-th point, for $1 \leq i < s$. Symmetrically, let the right-point of level 1 be the first point of another chain and the right-dominating point of

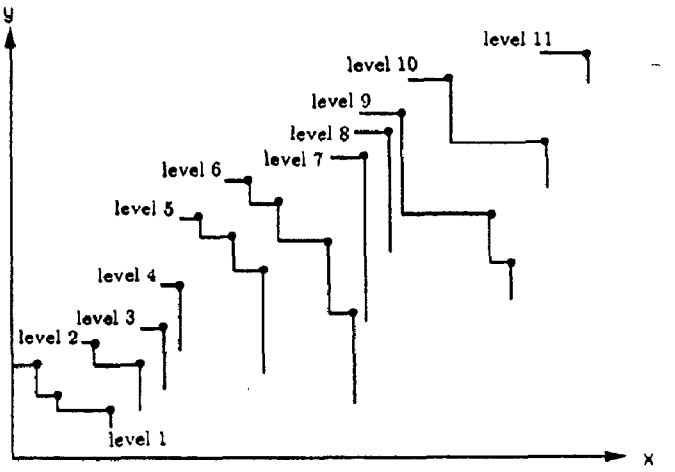


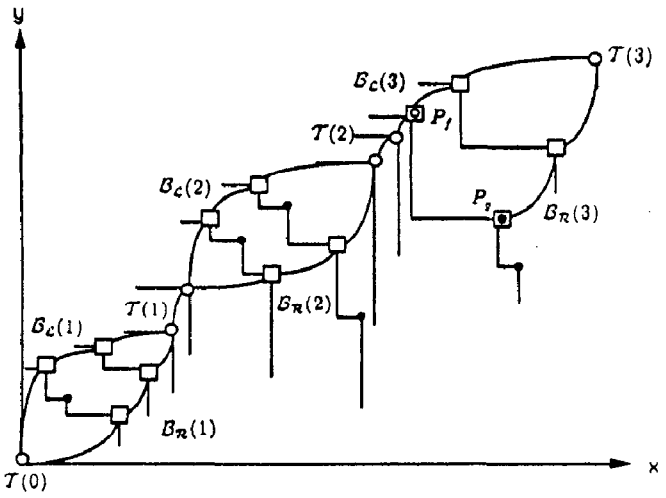
Figure 1 : To assign each point a level

the i-th point of the chain be the $(i + 1)$-th point, for $1 \leq i < s$. A chain with one point at each level is called a *solid chain.* Obviously, each of the two chains obtained above is a solid chain. If these two chains have no common point(s), then they form a maximum two-chain containing $2s$ points. If these two chains have some common point(s), then the union of these two chains may not be a maximum two-chain. However, the two chains, obtained above, will be used as a "skeleton" of the final maximum two-chain.

We assume level 0 contains the fictitious point $P_0 = (0, 0)$. Then we process the points from left to right, starting from level 0, level by level. For each level u we perform the following four phases :

(1) *local chain assignment phase,*
(2) *N-value assignment phase,*
(3) *candidate chain assignment phase,* and
(4) *adjustment phase.*

At the end we construct chains $\mathcal{T}(0)$,$\mathcal{B}_{\mathcal{L}}(1)$,$\mathcal{B}_{\mathcal{R}}(1)$, $\mathcal{T}(1)$,...,$\mathcal{B}_{\mathcal{L}}(k)$,$\mathcal{B}_{\mathcal{R}}(k)$, and $\mathcal{T}(k)$, where $k \leq s$, each of which is a subset of ρ: we refer to these chains as *local chains.* Chains $\mathcal{B}_{\mathcal{L}}(i)$, $\mathcal{B}_{\mathcal{R}}(i)$, and $\mathcal{T}(i)$ are called *left branch*, *right branch*, and *tie* of the ith local chains, respectively. A point in a local chain is called a *marked point*; otherwise, a point is *unmarked.* Initially, all local chains are empty. At local chain assignment phase of level 0, we assign P_0 to $\mathcal{T}(0)$. At local chain assignment phase of level u, where $u > 0$, we assign points at level u to local chains depending on the assignment of points to lo-



- : unmarked point
- : marked point assigned to a tie at local chain assignment phase
- : marked point assigned to a branch at local chain assignment phase
- : marked point assigned to a branch at adjustment phase
- : marked point assigned to a tie at local chain assignment phase and then removed to a branch at adjustment phase
- : local chains

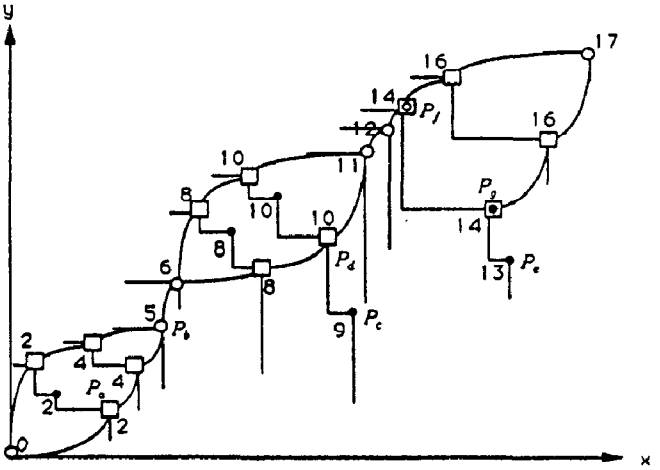
Figure 2 : Local chains

cal chains at level $u-1$ (see Figure 2). There are two cases :

LC1) Point P_a at level $u-1$ has been assigned to local chain $\mathcal{T}(i)$: If the left-dominating point and the right-dominating point of P_a are distinct, we assign them to $\mathcal{B}_{\mathcal{L}}(i+1)$ and $\mathcal{B}_{\mathcal{R}}(i+1)$, respectively. Otherwise, we assign the point dominating P_a to $\mathcal{T}(i)$.

LC2) Points P_a and P_b at level $u-1$ have been assigned to local chains $\mathcal{B}_{\mathcal{L}}(i)$ and $\mathcal{B}_{\mathcal{R}}(i)$, respectively: If the left-dominating point of P_a and the right-dominating point of P_b are distinct, then we assign them to $\mathcal{B}_{\mathcal{L}}(i)$ and $\mathcal{B}_{\mathcal{R}}(i)$, respectively; otherwise, we assign the point dominating both P_a and P_b to $\mathcal{T}(i)$.

At N-value assignment phase of level u, we assign a value $N(a)$ to each point P_a at level u, called *N-value* of P_a. Intuitively, $N(a)$ is the size of a maximum two-chain in $\rho_{1,L(a)}$ *containing* P_a (obviously, fictitious point P_0 does not contribute to any two-chains). Such a maximum two-chain is denoted by *P_a-max-2-chain*. We assign $N(a)$ to P_a according to the following rules (see Figure 3).



$C(a) = \{P_a\}$ $C(b) = \{P_b\}$ $C(c) = \{P_b, P_c\}$
$C(d) = \{P_d\}$ $C(e) = \{P_b, P_c, P_e\}$

Figure 3: N-value of each point and candidate chains of some points in Figure 1

R1) $N(0) = 0$ for P_0.

R2) If point P_a is in a tie, then $N(a)$ is equal to $N(b)+1$, where P_b is any marked point at level $L(a)-1$.

R3) If point P_a is in a branch, then $N(a)$ is equal to $N(b)+2$, where P_b is any marked point at level $L(a)-1$.

R4) If point P_a is unmarked, then $N(a)$ is equal to the maximal value of $[N(b)+L(a)-L(b)+1]$ over all points P_b dominated by P_a. One of the points, chosen arbitrarily, whose N- and L-values make $[N(b)+L(a)-L(b)+1]$ maximum is called the *preceding point* of P_a.

Note that marked points at the same level have the same N-value.

At candidate chain assignment phase of level u, for each point P_a at level u we construct a chain $C(a)$ with top P_a, called *candidate chain* of P_a, in the following manner (see Figure 3). If P_a is marked, then $C(a)=\{P_a\}$. Otherwise, $C(a) = C(b) \cup \{P_a\}$, where P_b is the preceding point of P_a. Note that the bottom of a candidate chain must necessarily be a marked point.

At adjustment phase of level u, we examine all points at that level in order to adjust local chains and N-values of marked points. If there exists a point P_a at level u belonging to a tie $\mathcal{T}(i)$, and there

also exists some unmarked point at level u, say P_b, whose N-value is the largest among all unmarked points at level u and is no less than $N(a)$, then we delete P_a from $\mathcal{T}(i)$ and set $N(a)$ equal to $N(b)$. If $x_b > x_a$ then we assign P_b to $\mathcal{B}_{\mathcal{R}}(i+1)$ and P_a to $\mathcal{B}_{\mathcal{L}}(i+1)$. Otherwise, we assign P_b to $\mathcal{B}_{\mathcal{L}}(i+1)$ and P_a to $\mathcal{B}_{\mathcal{R}}(i+1)$. Note that P_b is now a marked point but may have a candidate chain containing more than one point.

Local chains and candidate chains have properties stated in the following lemmas. Lemma 2 follows directly from local chain assignment rules LC1, LC2 and adjustment phase procedure.

Lemma 2 *At each level, there are exactly two points in branches, one in a left branch and the other in a right branch, or one point in a tie.*

Lemma 3 *There exists a solid chain from each marked point (or the marked point) at level u to a marked point at level v, $u < v$.*

Lemma 4 *Each marked point has the largest N-value among all points at the same level.*

Proof: Let us consider the following cases first:

(1) If P_p is the first point of a branch obtained at local chain assignment phase (from local chain assignment rule LC1), and P_q is the marked point (in a tie) at level $L(p)-1$, then $N(p) = N(q)+2 = N(q) + L(p) - L(q) + 1$.

(2) If two points P_p and P_q are in the same branch and $L(p) > L(q)$ (and hence P_p is assigned to the branch at local chain assignment phase), then from N-value assignment rule (repeated applications of rule R3), $N(p) = N(q)+2(L(p)-L(q)) \geq N(q) + L(p) - L(q) + 1$.

In other words, if P_p is a point assigned to a branch at local chain assignment phase, P_q is a marked point, and $L(p) > L(q)$, then $N(p) \geq N(q)+ L(p) - L(q) + 1$.

We now prove the lemma by induction on the *level* number. It is clear that the lemma is true for level 0. For level u, we assume the lemma is true for any level v, where $v < u$. Let P_a and P_b be a marked and an unmarked point at level u, respectively. We distinguish two cases:

Case 1) P_a is assigned to a branch at local chain assignment phase (see Figure 4): Let P_f be the preceding point of point P_b, and P_g be a marked point at level $L(f)$ (if P_f is marked, then let P_g be P_f). P_g may or may not be dominated by P_a. By inductive hypothesis, $N(g) \geq N(f)$. From the N-value assignment rule $R4$, $N(b)=N(f) + L(a) - L(g) + 1$ and from the previous discussion, $N(a) \geq N(g) + L(a) - L(g) + 1$. Therefore, $N(a) \geq N(b)$.

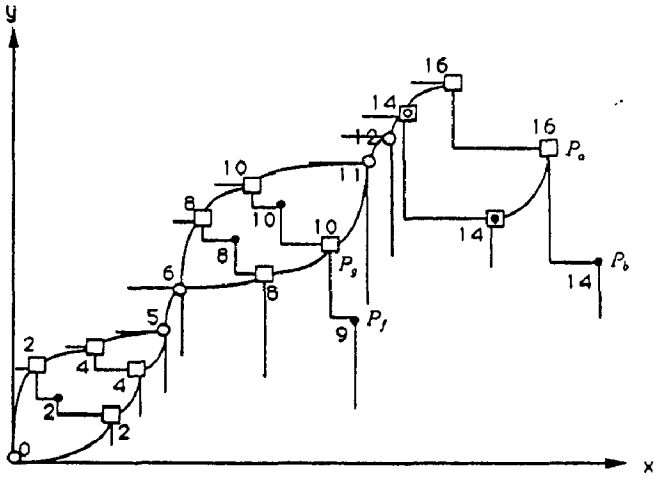


Figure 4 : Proof of Lemma 4

Case 2) P_a is a point in a tie or is assigned to a branch at adjustment phase: Because of the comparison of N-values at adjustment phase, it is clear that the lemma is true in this case.

Hence, the lemma is established. □

In the following, we show how to construct a two-chain $\mathcal{M}(a)$ associated with a point P_a ($\mathcal{M}'(a) = \mathcal{M}(a) - \{P_0\}$ being a P_a-max-2-chain; see Lemma 5 below).

M1) For P_0, $\mathcal{M}(0) = \{P_0\}$.

M2) To construct $\mathcal{M}(a)$ of P_a, where $L(a) > 0$, we assume $\mathcal{M}(g)$ of each point P_g at level $L(g)$, where $L(g) < L(a)$, has been constructed. We have the following cases:

M2.1) P_a is an unmarked point (see Figure 5a): Let the marked point(s) at level $L(a)$ be P_b (and $P_{b'}$), P_d the *preceding point* of P_a at level $L(d) < L(a)$, and P_e a marked point at level $L(e) = L(d)$. According to Lemma 3, there exists a solid chain from P_e to P_b (or from P_e to $P_{b'}$). We assign to $\mathcal{M}(a)$ the union of

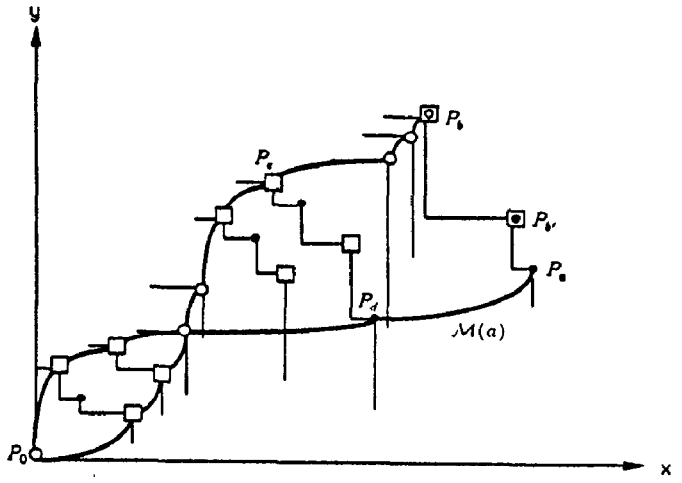


Figure 5a : $\mathcal{M}(a)$ of an unmarked point P_a

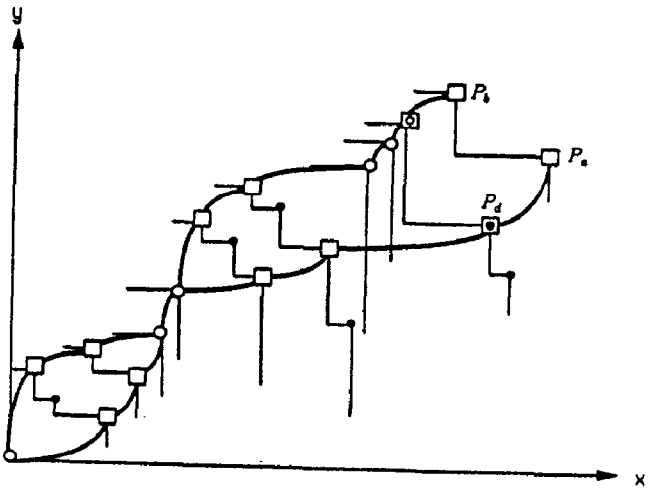


Figure 5c: $\mathcal{M}(a)$ of a point P_a in a branch

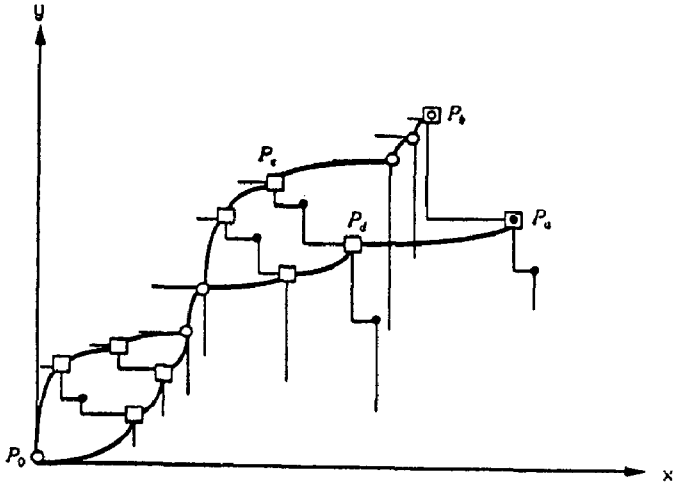


Figure 5b: $\mathcal{M}(a)$ of a point P_a in a branch

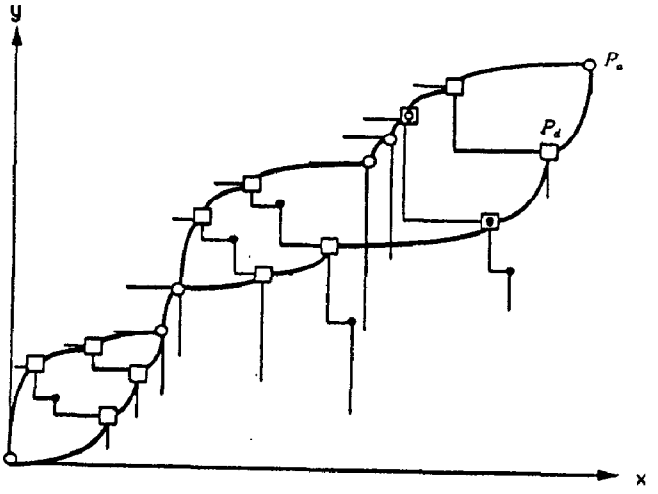


Figure 5d: $\mathcal{M}(a)$ of a point P_a in a tie

$\mathcal{M}(d)$, $\{P_a\}$, and solid chain from P_e to P_b (or $P_{b'}$).

M2.2) P_a is a marked point in a tie or a branch:

(1) If $P_a \in \mathcal{B}_{\mathcal{R}}(i)$ and $P_b \in \mathcal{B}_{\mathcal{L}}(i)$ are the marked points at level $u = L(a) = L(b)$, and one of them, say P_a, has a preceding point P_d at level $L(d) < u$ (see Figure 5b), then we assign to $\mathcal{M}(a)$ the set $\mathcal{M}(d) \cup \{P_a\} \cup \mathcal{C}$, where $\mathcal{C}$ denotes a solid chain from a marked point P_e at level $L(e) = L(d)$ to point P_b.

(2) If $P_a \in \mathcal{B}_{\mathcal{R}}(i)$ and $P_b \in \mathcal{B}_{\mathcal{L}}(i)$ are the marked points at level $u = L(a) = L(b)$, and both of them have no preceding point (see Figure 5c), we assign to $\mathcal{M}(a)$ the set $\mathcal{M}(d) \cup \{P_a, P_b\}$, where P_d denotes a marked point at level $L(d) = L(a) - 1$.

(3) If P_a is in a tie, let P_d be a marked point at level $L(d) = L(a) - 1$ (see Figure 5d). We assign to $\mathcal{M}(a)$ the set $\mathcal{M}(d) \cup \{P_a\}$.

Lemma 5 *For each point P_a, $\mathcal{M}'(a)$ is a P_a-max-2-chain containing $N(a)$ points.*

Proof: This can be proved by induction on the *level* number. We refer the reader to the full paper [LSL] for details.

The following theorem is readily established from Lemmas 4 and 5.

Theorem 1 *If P_a is a marked point at level s, then P_a-max-2-chain is a maximum two-chain in ρ.*

$\mathcal{M}(a) - \{P_0\}$ in Figure 5d is a maximum two-chain of the point set in Figure 1.

3 Details of Implementation

We summarize the entire algorithm for finding a maximum two-chain below. Algorithm LEVEL computes the levels of the points in ρ using plane sweep approach [PS] and takes $O(n \log n)$ time. Algorithm READ-MAX-TWO-CHAIN finds $\mathcal{M}(a)$, and hence $\mathcal{M}'(a)$, of a marked point P_a at level s and takes linear time. According to Lemma 5 and Theorem 1, $\mathcal{M}'(a)$ is a maximum two-chain in ρ. The remaining algorithms correspond to the *four* phases described above. They, except algorithm N-ASSIGNMENT, take linear time. We elaborate below the algorithm N-ASSIGNMENT, which is most complicated.

Procedure MAX-TWO-CHAIN (ρ)
  begin
    call LEVEL(ρ) ;
    for $k = 0$ to s
      begin
        call LOCAL-CHAIN(k) ;
        call N-ASSIGNMENT(k) ;
        call C-CHAIN(k) ;
        call ADJUSTMENT(k) ;
      end
    call READ-MAX-TWO-CHAIN(ρ) ;
  end.

Algorithm N-ASSIGNMENT

Algorithm N-ASSIGNMENT(k) assigns a value $N(a)$ to each point P_a at a specified level k. If P_a is a marked point, it is trivial to assign $N(a)$ according to assignment rules R1, R2, and R3. If P_a is unmarked then, according to R4, it is necessary to find a point P_b dominated by P_a and with maximum $[N(b) - L(b) + 1]$. Rule R4 can be done in an obvious way: for each point P_a we examine exhaustively all points dominated by P_a and find a point P_b with the maximum $[N(b) - L(b) + 1]$. But this would take $O(n)$ time, resulting in an $O(n^2)$ time algorithm. We shall show below that with a careful manipulation of the value $W(b) = N(b) - L(b)$, referred to as the *weight* of point P_b, we can find for each point P_a the desired point in $O(\log n)$ time. Toward this end we introduce the notion of *staircases* associated with the points in ρ.

3.1 Staircases

In order to simplify the discussion, we *assume* that $|\rho_k| = r_k$, $k = 1, 2, \ldots, s$, and the points in ρ are reindexed as $\rho_1 = \{P_1, P_2, \ldots, P_{r_1}\}$, $\rho_2 = \{P_{r_1+1}, \ldots, P_{r_1+r_2}\}, \ldots$, $\rho_s = \{P_{r_1+\cdots+r_{s-1}+1}, \ldots, P_{r_1+\cdots+r_s}\}$ so that $x_1 < x_2 < \cdots < x_{r_1}$, $x_{r_1+1} < x_{r_1+2} < \ldots < x_{r_1+r_2}$, etc. Consider the points in ρ_1. Let $P_{i,d} = (x_i, 0)$ be the *downward vertical intersection* (d-intersection for short) of P_i, $i = 1, 2, ..., r_1$. Let $P_{1,l} = (0, y_1)$ and $P_{i,l} = (x_{i-1}, y_i)$ be the *leftward horizontal intersection* (l-intersection for short) of the left point P_1 and other points P_i in ρ, respectively, on the y-axis and the vertical line segment $\overline{P_{i-1}, P_{i-1,d}}$, $i = 2, 3, \ldots, r_1$ (see Figure 6a). The rectilinear path $(P_{1,l} P_1 P_{2,l} P_2 \ldots P_{i,l} P_i P_{i,d})$, abbreviated as $(P_{1,l} \ldots P_{i,d})$, is referred to as the *staircase* $\mathcal{S}(P_i)$, $i = 1, 2, \ldots, r_1$. For convenience we augment the staircase by two points, $P_H = (0, \infty)$ and $P_T = (\infty, 0)$, and denote the staircase as $(P_H P_{1,l} P_1 \ldots P_i P_{T,l} P_T)$, where $P_{T,l} := P_{i,d}$. Furthermore, we adopt the convention that each point P_j in $\rho \cup \{P_H, P_T\}$ on a staircase has both d- and l-intersections on the staircase except P_H, which has only d-intersection $P_{H,d} := P_{1,l}$, and P_T, which has only l-intersection $P_{T,l} := P_{i,d}$. That is, we shift each $P_{j,d}$ up to the staircase and let new $P_{j,d} := P_{j+1,l}$, where $j = 1, 2, ..., i-1$. Staircase $\mathcal{S}(P_{r_1})$ is called the staircase of level 1, denoted $\mathcal{S}_1$, and is shown in Figure 6a.

Consider now the points in ρ_2, i.e., $P_{r_1+1}, P_{r_1+2}, \ldots, P_{r_1+r_2}$. Imagine dropping downward a vertical line V_i from each point P_i, $r_1 < i \leq r_1 + r_2$. The point in $\mathcal{S}_1$ that is hit by V_i is the d-intersection, denoted by $P_{i,d}$, of P_i on $\mathcal{S}_1$ (see Figure 6b). For $P_i \in \rho_2$, the l-intersection on $\mathcal{S}_1$, $P_{i,l}$, is defined in the same way as for the points in ρ_1. Let P_a be the left point in ρ_2. The l- and d-intersections, $P_{a,l}$ and $P_{a,d}$,

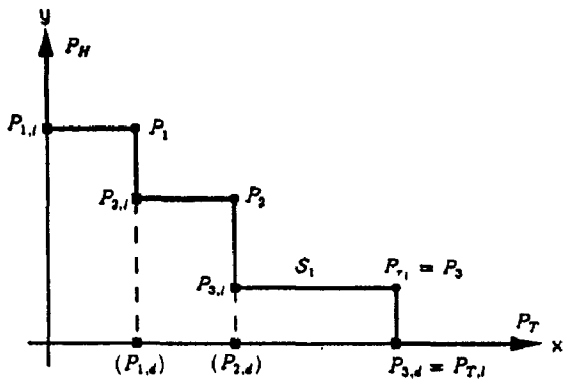


Figure 6a: Staircase S_1 (where $r_1 = 3$)

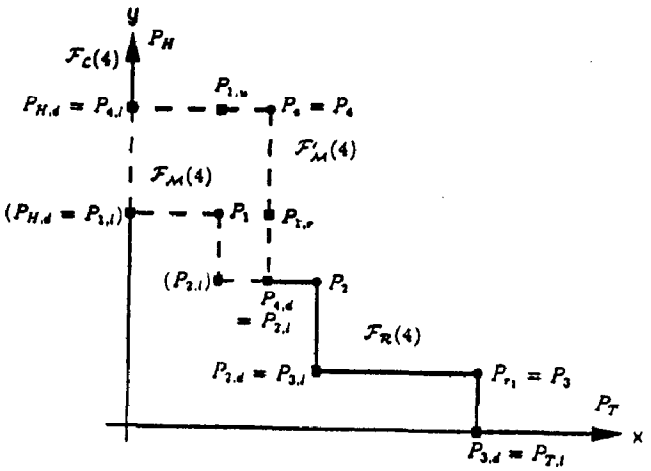


Figure 6b: Staircase $\mathcal{S}(P_4)$ (where $r_1 = 3$)

respectively, partition $\mathcal{S}_1$ into three *substaircases*: $\mathcal{F}_{\mathcal{L}}(P_a), \mathcal{F}_{\mathcal{M}}(P_a)$, and $\mathcal{F}_{\mathcal{R}}(P_a)$, where $\mathcal{F}_{\mathcal{L}}(P_a) = (P_H P_{a,l})$ is the *leading portion* of $\mathcal{S}_1$, $\mathcal{F}_{\mathcal{M}}(P_a) = (P_{a,l} \ldots P_{a,d})$, is the *middle portion* of $\mathcal{S}_1$ and $\mathcal{F}_{\mathcal{R}}(P_a) = (P_{a,d} \ldots P_{T,l} P_T)$ is the *trailing portion* of $\mathcal{S}_1$. Let $P_{k,l}$ and P_k be two consecutive points in $\mathcal{S}_1$ such that $x_{k,l} < x_a < x_k$. The *upward vertical projection* (u-projection for short) of all the points excluding the endpoints $P_{a,l}$ and $P_{a,d}$ in $\mathcal{F}_{\mathcal{M}}(P_a)$ on the horizontal line segment $\overline{P_{a,l}, P_a}$ gives rise to the u-projections, $P_{1,u}, P_{2,u}, \ldots, P_{k-1,u}$. Similarly, the *rightward horizonal projection* (r-projection for short) of all the points excluding the two endpoints in $\mathcal{F}_{\mathcal{M}}(P_a)$ on the vertical line segment $\overline{P_a, P_{a,d}}$ yields r-projections, $P_{1,r}, P_{2,r}, \ldots, P_{k-1,r}$. The staircase $\mathcal{S}(P_a)$ associated with P_a is obtained from $\mathcal{S}_1$ as the concatenation of $\mathcal{F}_{\mathcal{L}}(P_a)$, $\mathcal{F}'_{\mathcal{M}}(P_a)$ and $\mathcal{F}_{\mathcal{R}}(P_a)$, where $\mathcal{F}'_{\mathcal{M}}(P_a)$ is the "substaircase" $(P_{a,l} P_{1,u} \ldots P_{k-1,u} P_a P_{1,r} \ldots P_{k-1,r} P_{a,d})$ derived from projecting $\mathcal{F}_{\mathcal{M}}(P_a)$ onto $\overline{P_{a,l}, P_a}$ and $\overline{P_a, P_{a,d}}$. Thus $\mathcal{S}(P_a)$ contains not only point P_a, its l- and d-intersections $P_{a,l}$ and $P_{a,d}$, but also the leading and trailing portions of $\mathcal{S}_1$ and projections of the points in the middle portion of $\mathcal{S}_1$.

Let P_a be the left point in $\rho_j, j > 1$. In general, the staircase $\mathcal{S}(P_i)$ of any point $P_i \in \rho_j$, is a rectilinear path of the form $(P_H\ P_{a,l}\ \mathcal{U}_a\ P_a\ \mathcal{R}_a\ P_{a+1,l}\ \mathcal{U}_{a+1}\ P_{a+1}\ \mathcal{R}_{a+1}\ \ldots\ P_{i,l}\ \mathcal{U}_i\ P_i\ \mathcal{R}_i\ P_{i,d}\ \mathcal{F}_{\mathcal{R}}(P_i))$, where the $\mathcal{U}$'s, $\mathcal{R}$'s, and $\mathcal{F}_{\mathcal{R}}(P_i)$ are respectively, u-projections, r-projections, and the trailing portion of the previous staircase $\mathcal{S}(P_{i-1})$. Note that, $\mathcal{F}_{\mathcal{L}}(P_i) = (P_H\ P_{a,l}\ \mathcal{U}_a\ P_a\ \mathcal{R}_a\ P_{a+1,l}\ \mathcal{U}_{a+1}\ P_{a+1}\ \mathcal{R}_{a+1}\ \ldots\ P_{i,l})$ and $\mathcal{F}_{\mathcal{M}}(P_i) = (P_{i,l}\ \mathcal{U}_i\ P_i\ \mathcal{R}_i\ P_{i,d})$. Now let us consider how to obtain $\mathcal{S}(P_{i+1})$ from $\mathcal{S}(P_i)$.

We first find the l-intersection $P_{i+1,l}$ on VL$=\overline{P_i, P_{i,d}}$ and the d-intersection $P_{i+1,d}$ on $\mathcal{S}(P_i)$. Let R' and R'' be two consecutive points on VL containing $P_{i+1,l}$, and let Q' and Q'' be two consecutive points on $\mathcal{S}(P_i)$ containing $P_{i+1,d}$. Thus $\mathcal{F}_{\mathcal{M}}(P_{i+1})$ is the substaircase $(R'' \ldots Q')$ of $\mathcal{S}(P_i)$ and the new $\mathcal{F}'_{\mathcal{M}}(P_{i+1})$ is obtained by projecting all points (including projections) in $\mathcal{F}_{\mathcal{M}}(P_{i+1})$ vertically upward and horizontally rightward onto $\overline{P_{i+1,l}, P_{i+1}}$, $\overline{P_{i+1}, P_{i+1,d}}$, respectively. Note that, points on the same vertical line segment have the same u-projection and points on the same horizontal line segment have the same r-projection. The staircase $\mathcal{S}(P_{i+1})$ is the concatenation of the leading portion $(P_H \ldots R')$ of $\mathcal{S}(P_i)$, $\mathcal{F}'_{\mathcal{M}}(P_{i+1})$, and the trailing portion $(Q'' \ldots P_T)$ of $\mathcal{S}(P_i)$. According to the convention adopted, we shift certain points each time a new staircase is obtained. Specificaly, let $P_f \in \rho \cup \{P_H, P_T\}$ be the rightmost point on $\mathcal{S}(P_i)$ whose y-coordinate is greater than y_{i+1}, and let $P_g \in \rho \cup \{P_H, P_T\}$ be the leftmost point on $\mathcal{S}(P_i)$ whose x-coordinate is greater than x_{i+1}. We let $P_{i+1,l}$ and $P_{i+1,d}$ be the new $P_{f,d}$ and $P_{g,l}$ respectively; that is, we shift $P_{f,d}$ and $P_{g,l}$ from their

original positions to $P_{i+1,l}$ and $P_{i+1,d}$, respectively. For example, in Figure 6b, when $\mathcal{S}(P_4)$ is obtained, $P_{2,l}$ is shifted to $P_{4,d}$ and $P_{H,d}$ is shifted to $P_{4,l}$.

3.2 Implementation of algorithm N-ASSIGNMENT

Clearly, for every point at level 1, its N-value equals 1 if $|\rho_1| = 1$, or 2 if $|\rho_1| > 1$. Recall that the staircase $\mathcal{S}(P_i)$ of any point $P_i \in \rho_j$, $j > 1$ is a rectilinear path of the form (P_H $P_{a,l}$ $\mathcal{U}_a$ P_a $\mathcal{R}_a$ $P_{a+1,l}$ $\mathcal{U}_{a+1}$ P_{a+1} $\mathcal{R}_{a+1}$ ... $P_{i,l}$ $\mathcal{U}_i$ P_i $\mathcal{R}_i$ ... $\mathcal{U}_q$ P_q $\mathcal{R}_q$ $P_{T,l}$ P_T), where P_a is the left point in ρ_j and P_q is the right point in ρ_{j-1}. We now describe how the N-value of each point P_i is computed and hence the weight $W(i) = N(i) - L(i)$.

We maintain a 2-3 tree for all the points P_H, $P_{i,l}$, P_i in ρ, $P_{T,l}$, and P_T of $\mathcal{S}(P_i)$ such that each leaf corresponds to a point. The leaf v corresponding to P_i in ρ is the root of a binary tree so that the left and right subtrees, denoted by $\mathcal{U}(P_i)$ and $\mathcal{R}(P_i)$, are height balanced and contain u-projections $\mathcal{U}_i$ and r-projections $\mathcal{R}_i$, respectively. The leaf corresponding to $P_{i,l}$, where $P_i \in \rho$, stores the maximum weight of the points dominated by $P_{i,l}$. The leaves corresponding to P_H, $P_{T,l}$, and P_T store weight 0. Each leaf w in the height balanced subtrees corresponds to a r- or u-projection of a point P_w or its projection and stores the weight of the point P_w. The height balanced subtrees are maintained so that at each internal node w a weight $W(w)$, which is equal to the maximum weight among those stored at the leaves rooted at w, is stored. This is referred to as the *max heap* property. The points in these trees are all maintained according to the order in which they are traversed along the staircase. Initially, the two height-balanced subtrees $\mathcal{U}(P_i)$ and $\mathcal{R}(P_i)$ at node v_i corresponding to point P_i in ρ_1 are empty reflecting the fact that the $\mathcal{U}$'s and $\mathcal{R}$'s are empty. All leaves in the 2-3 tree, except the leaves corresponding to points $P_i \in \rho_1$ which contain the weight $N(i) - 1$, are assigned weight equal to 0.

We now describe how to maintain the 2-3 tree and the height-balanced trees as we process the points in a level-by-level left-to-right order. Consider the staircase $\mathcal{S}(P_i)$ as described above and let the next point to be processed is P_{i+1}. We first perform a search in the 2-3 tree to find the two points P_a and P_b in $\rho \cup \{P_H, P_T\}$ that satisfy the following:

(i) P_a is the rightmost point whose y-coordinate is greater than y_{i+1}.

(ii) P_b is the leftmost point whose x-coordinate is greater than x_{i+1}.

Points in $\mathcal{R}(P_a)$ and $\mathcal{U}(P_b)$, $P_{q,l}$'s, P_q's, and all points in $\mathcal{U}(P_q)$ and $\mathcal{R}(P_q)$, where $P_q \in \rho_{L(i+1)-1}$, that are dominated by P_{i+1}, are in $\mathcal{F}_{\mathcal{M}}(P_{i+1})$. We find the point P_c in $\mathcal{F}_{\mathcal{M}}(P_{i+1})$ with the maximum weight $W(c)$ and assign to P_{i+1} the appropriate N-value. To be more specific, we perform the following steps on the trees while maintaining the *max heap* property of the height balanced subtrees.

1. The leaves of the 2-3 tree that lie between P_a and P_b are deleted and replaced by three leaves corresponding to $P_{i+1,l}$, P_{i+1}, and $P_{i+1,d}$.

2. Split the tree $\mathcal{R}(P_a)$ into two subtrees $\mathcal{R}_l(P_a)$ and $\mathcal{R}_r(P_a)$ such that the (projection) points in $\mathcal{R}_l(P_a)$ have y-coordinates greater than y_{i+1} and those in $\mathcal{R}_r(P_a)$ have y-coordinates less than y_{i+1}. Replace $\mathcal{R}(P_a)$ with $\mathcal{R}_l(P_a)$.

3. Split the tree $\mathcal{U}(P_b)$ into two subtrees $\mathcal{U}_l(P_b)$ and $\mathcal{U}_r(P_b)$ such that the (projection) points in $\mathcal{U}_l(P_b)$ have x-coordinates less than x_{i+1} and those in $\mathcal{U}_r(P_b)$ have x-coordinates greater than x_{i+1}. Replace $\mathcal{U}(P_b)$ with $\mathcal{U}_r(P_b)$.

4. The weight of the node corresponding to $P_{i+1,l}$ (resp. $P_{i+1,d}$) is the maximum of those in $\mathcal{R}_r(P_a)$ and that in $P_{a,d}$ (resp. $\mathcal{U}_l(P_b)$ and $P_{b,l}$).

5. The left subtree $\mathcal{U}(P_{i+1})$ is the concatenation of the trees $\mathcal{U}_l(P_b)$ and all $\mathcal{U}(P_q) \cup \{P_q\}$, where P_q's are as defined above.

6. The right subtree $\mathcal{R}(P_{i+1})$ is the concatenation of the trees $\mathcal{R}_r(P_a)$ and all $\mathcal{R}(P_q) \cup \{P_q\}$, where P_q's are as defined above.

7. If P_{i+1} is marked, we assign N-value to P_{i+1} according to assignment rules R2 and R3. Otherwise, we find the point P_c in $\{P_{i+1,l}\} \cup \mathcal{U}(P_{i+1}) \cup \mathcal{R}(P_{i+1}) \cup \{P_{i+1,d}\}$ with the maximum weight $W(c)$ and assign $W(c) + L(i+1) + 1$ to $N(i+1)$. The weight $W(i+1)$ is $N(i+1) - L(i+1)$.

We have an important property of the staircase $\mathcal{S}(P_a)$, referred to as the *projection containment property* (Lemma 6). Note that the u-projection of a u-projection of point P_i is considered a u-projection of P_i (i.e., $P_{i,u}$); the r-projection of point P_i is similarly defined. The u-(or r-)projection of a r-(or u-)projection of a point is simply called a projection of the point. For simplicity, the u-projection and the r-projection of point P_i are also called the projections of the point.

A point P_i dominated by P_j is said to satisfy the *projection containment property*, if the following are true: (1) If the x-coordinate of $P_{j,l}$ is greater than x_i, then a projection of P_i appears at $P_{j,l}$; otherwise, at least a projection $P_{i,u}$ appears on $\overline{P_{j,l}P_j}$. (2) If the y-coordinate of $P_{j,d}$ is greater than y_i, then a projection of P_i appears at $P_{j,d}$; otherwise, at least a projection $P_{i,r}$ appears on $\overline{P_jP_{j,d}}$.

Lemma 6 *Each point P_i in ρ satisfies the projection containment property with respect to each point P_j dominating P_i in ρ.*

Proof: Assume the points domonating P_i in ρ are $P_{\alpha_1}, P_{\alpha_2}, \ldots, P_{\alpha_z}$, where z is the number of points in ρ dominating P_i and $\alpha_1 < \alpha_2 < \ldots < \alpha_z$. We prove the lemma inductively. For inductive basis, consider P_{α_1} first. P_{α_1} must be the left-dominating point of P_i. Obviously, $P_{i,u}$ and $P_{i,r}$ are in $\mathcal{F}'_{\mathcal{M}}(P_{\alpha_1})$, and the lemma is true for P_i with respect to P_{α_1}.

Now consider point P_{α_h}, and suppose inductively that the lemma is true for P_i with respect to each point P_{α_k}, where $k < h$. There are two cases:

Case 1) If $P_{\alpha_{(h-1)}}$ and P_{α_h} are at the same level then, by definition, either P_i has a projection at $P_{\alpha_{(h-1)},d}$ or $P_{i,r}$ appears at a position on $\overline{P_{\alpha_{(h-1)}}P_{\alpha_{(h-1)},d}}$ depending on if y_i is less than or greater than the y-coordinate of $P_{\alpha_{(h-1)},d}$, respectively. Thus, P_i will have a projection at $P_{\alpha_h,l}$.

Case 2) If $P_{\alpha_{(h-1)}}$ and P_{α_h} are not at the same level, then let P_b be the leftmost point of $\mathcal{S}(P_{\alpha_{h-1}})$ in ρ dominating P_i. By induction hypothesis, there is a u-projection of P_i, i.e., $P_{i,u}$, that lies on $\overline{P_{b,l}P_b}$. The x-coordinate of $P_{i,u}$ is less than x_{α_h}, and thus, $P_{i,u}$ will be projected on $\overline{P_{\alpha_h,l}P_{\alpha_h}}$ in $\mathcal{S}(P_{\alpha_h})$, for there are no points between $P_{\alpha_{(h-1)}}$ and P_{α_h} dominating P_i.

Similarly, we can show that either P_i has a projection at $P_{\alpha_h,d}$, or $P_{i,r}$ appears on $\overline{P_{\alpha_h}P_{\alpha_h,d}}$ in $\mathcal{S}(P_{\alpha_h})$. Thus, the lemma is true. □

We now analyze the complexity of algorithm N-ASSIGNMENT. Step 1 can be done in $O(\log n)$ time per point. Steps 2-7 require SPLIT and SPLICE operations, each taking $O(\log n)$ time [AHU]. Let $P_i \in \rho_k$ be the left-dominating point of some points P_w's $\in \rho_{k-1}$. The number of times the operation SPLICE is executed is equal to twice the number of points P_w. Since a point P_w can appear at most once as a leaf in the 2-3 tree, when it is inserted and is "left-dominated" by only one point at the next higher level, the total amount of time contributed by each point is at most $O(\log n)$. Thus, algorithm N-ASSIGNMENT takes $O(n \log n)$ time.

It is obvious that the space requirement is only linear, and we thus have the following main result.

Theorem 2 *Algorithm MAX-TWO-CHAIN finds a maximum two-chain in a point set ρ in $\Theta(n)$ space and $O(n \log n)$ time, where $n = |\rho|$.*

In the following, we show that $\Omega(n \log n)$ is a time lower bound for the maximum two-chain problem. In [KR] it is shown that the problem of finding the set of points on the dominance hull of a given set ρ of n points requires $\Omega(n \log m)$ time under the algebraic decision (computation) tree model (see [AHU,BO]), where m is the number of maximal points in ρ. We can apply the same argument to the problem of finding a maximum chain and show that $\Omega(n \log m)$ time is required to find a maximum chain of m points. In the worst case, since the number of points on the dominance hull can be n, $\Omega(n \log n)$ is a lower bound for the maximum one chain problem.

We now show that the maximum two-chain problem requires $\Omega(n \log n)$ time as well. Given a point set ρ, we can construct in linear time a chain $\bar{\rho}$, in which each point is crossing with all points in ρ . A maximum two-chain of $\rho \cup \bar{\rho}$ should include $\bar{\rho}$ and a maximum chain of ρ. So we can find a maximum chain of ρ by applying any maximum two-chain algorithm to $\rho \cup \bar{\rho}$. The following theorem is established.

Theorem 3 *In the algebraic computation tree model, any algorithm that finds a maximum two-chain in a point set ρ requires $\Omega(n \log n)$ time, where $n = |\rho|$.*

4 Conclusion

We have presented an optimal algorithm that runs in $\Theta(n \log n)$ time and $\Theta(n)$ space for solving the maximum two chain problem : given a set ρ of n points, find a maximum subset $\mathcal{C} \subseteq \rho$ of points such that $\mathcal{C}$ can be divided into two chains, each of which contains points that dominate each other. Whether or not the algorithm can be extended to handle weighted point set version or restricted versions of the problem (e.g., when some points are to be assigned entirely to chain i, $i = 1, 2$) remains to be seen. Both classes of problems have been previously solved in $O(n^2 \log n)$ time [SL1,SL2].

References

[AHU] Aho, A.V., Hopcroft, J.E., and Ullman, J.D., *The Design and Analysis of Computer Algorithms*, Addison Wesley, 1974.

[AK] Atallah, M.J., Kosaraju, S.R., "An Efficient Algorithm for Maxdominance, with Applications," *Algorithmica*, Vol. 4, No. 2, pp. 221-236, 1989.

[BO] Ben-Or, M., "Lower Bounds for Algebraic Computation Trees," *Proc. 15th ACM Symp. on Theory of Computing,* Boston, Mass., pp. 80-86, 1983.

[KR] Kapoor, S. and P. Ramanan, "Lower Bounds for Maximal and Convex Layers Problems," *Algorithmica*, to appear.

[PS] Preparata, F.P. and Shamos, M.I., *Computational Geometry,* Springer-Verlag, New York, 1985.

[SL1] Sarrafzadeh, M. and Lee, D.T., "A New Approach to Topological Via Minimization," *IEEE Trans. on CAD*, Vol. 8, No. 8, pp. 890-900, August 1989.

[SL2] Sarrafzadeh, M. and Lee, D.T., "Topological Via Minimization Revisited," Technical Report, CIMS 88-01, Northwestern University, November 1988.